Sensitivity Label Migration Project — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Sensitivity Label Migration Project (Microsoft Purview) 5

The problem . . . 5

The solution . . . 5

How it works . . . 5

What is delivered . . . 5

Business outcome . . . 5

Important boundaries . . . 6

1. Solution Overview — Sensitivity Label Migration Project (Microsoft Purview) 6

What this solution is . . . 6

Why this matters . . . 7

How it works . . . 7

The migration mapping (what the matrix covers) . . . 8

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 9

The standard deployment model . . . 9

2. Requirements Guide — Sensitivity Label Migration Project 9

2.1 Licensing for auto-labeling (the reclassification engine) . . . 10

2.2 The target sensitivity labels already exist and are published . . . 10

2.3 A legacy classification export from the source tool . . . 11

2.4 Detectability confirmed per legacy value . . . 11

2.5 Information Protection role (least privilege) . . . 11

2.6 Security & Compliance PowerShell + Graph connectivity . . . 12

2.7 A pilot scope and a simulation-first commitment . . . 12

2.8 A residual-marking reconciliation decision . . . 12

Global Requirements — Microsoft Purview Solutions 13

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 13

2. Purview roles (least-privilege) and portal access . . . 13

3. Security & Compliance PowerShell + Graph toolchain . . . 14

4. Data sources & scope in scope . . . 14

5. Compliance, records, privacy & legal stakeholders . . . 14

6. A pilot scope and change control . . . 14

3. Implementation Guide — Sensitivity Label Migration Project 15

Step 1 — Inventory both sides (Assess) . . . 15

Step 2 — Build the mapping matrix and choose detection (Design) . . . 16

Step 3 — Configure reclassification in simulation (Configure, preview-first) . . . 16
Step 4 — Run simulation and measure precision (Pilot) . . . 17
Step 5 — Validate coverage and precision (Validate) . . . 17
Step 6 — Enforce from simulation to reclassify (Expand — pilot locations) . . . 17
Step 7 — Expand location-by-location and decommission the source tool (Expand) . . . 18
5. Testing & Validation Guide — Sensitivity Label Migration Project 18
Test 1 — Both sides inventoried and cross-referenced . . . 19
Test 2 — Mapping matrix is complete . . . 19
Test 3 — Detection SITs exist and match a sample . . . 19
Test 4 — Reclassification policies are created in simulation only . . . 19
Test 5 — Simulation has scanned and produced a match report . . . 20
Test 6 — Reclassification precision is acceptable (the enforcement gate) . . . 20
Test 7 — Enforced reclassification matches the accepted simulation . . . 20
Test 8 — Encryption/marking on a mapped label behaves as designed . . . 20
Test 9 — Residual legacy marking is reconciled as decided . . . 21
Test 10 — Coverage & mapping-health check before expansion . . . 21
6. Troubleshooting — Sensitivity Label Migration Project (Microsoft Purview) 21
Licensing & connectivity . . . 21
Inventory & mapping (Steps 1–2) . . . 22
Creating simulation policies (Step 3) . . . 22
Running simulation & measuring precision (Step 4) . . . 22
Coverage & health (Step 5) . . . 22
Enforcing & rollback (Steps 6–7) . . . 23
What this solution does NOT decide . . . 23
7. Best Practices — Sensitivity Label Migration Project 23
Mapping design . . . 23
Detection (the real risk) . . . 24
Simulation & precision . . . 24
Enforcement & protection . . . 24
Reconciliation & decommission . . . 25
Program & change management . . . 25
Security & safety hygiene . . . 25
8. Deliverables Summary — Sensitivity Label Migration Project 25
What you receive . . . 26
Documentation . . . 26
PowerShell tools (5) . . . 27

Outcomes 27
Explicitly out of scope (deferred) 28
Appendix A — PowerShell Scripts 29

Executive Summary — Sensitivity Label Migration Project (Microsoft Purview)

The problem

Most organizations classified their content **before** Microsoft Purview — with an on-premises or third-party classification tool, or an older Microsoft labeling client. Those legacy markings are not Purview sensitivity labels: they don't drive Purview encryption, DLP, retention, or Copilot data protection, and they leave the estate running **two classification schemes at once**. That is a compliance gap (governance keyed off labels the platform can't read) and a data-risk exposure (content the new controls can't see) — and legacy labels **do not convert themselves**; there is no supported button that turns a third-party marking into a Purview label.

The solution

A complete, self-guided project that **maps** every legacy/third-party classification value to a target Microsoft Purview **sensitivity label** and **reclassifies** the content onto those labels. It builds a detection for each legacy value, an auto-labeling policy that applies the mapped label wherever that detection fires, and — critically — runs the whole reclassification **in simulation first**, measuring precision before a single file is touched. It uses only the Microsoft Purview portal and Security & Compliance PowerShell — no third-party tooling on the target side.

How it works

The engagement follows one disciplined model: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce).** Inventory both sides — the legacy values (from the source tool's export) and the target Purview estate; build a **mapping matrix** (each legacy value → one existing label) and choose a **detection** per value; configure the reclassification auto-labeling **in simulation**; let it scan and **measure precision** with the data owner; validate coverage; then **enforce from simulation** to reclassify, expanding location-by-location. Five PowerShell tools — read-only or `-WhatIf` by default — automate the two-sided inventory, the simulation-first configuration, the coverage/precision reporting, and the health checks.

What is delivered

- Six implementation documents (Overview, Requirements, Implementation, Testing, Best Practices, Troubleshooting) plus this summary.
- Five Security & Compliance PowerShell tools: two-sided inventory, reclassification config/remediation helper (simulation-first), simulation & health helper, coverage & precision report, and a mapping-health check.
- A named, gated, reversible migration with explicit pass/fail criteria — and a precision sign-off gate before any content is reclassified.

Business outcome

One classification scheme the platform actually governs. Legacy markings are mapped and content is reclassified onto Microsoft Purview labels — proven in simulation before enforcement, so nothing is mislabeled and no one is surprised by new encryption. Purview's controls (DLP, retention, Copilot protection) can then key off labels the platform understands — at a fraction of the cost of a bespoke consulting engagement.

Important boundaries

Whether a mapped label should **encrypt** or reproduce the legacy tool's protection, and whether a value is a "record," are **determinations owned by your records, security, compliance, and legal functions** — this solution provides the mapping method, detection, and evidence, not the determination. The reclassification engine (service-side auto-labeling) is **licensing-dependent** (E5 / E5 Compliance), the destination labels are assumed to already exist, and whether a given legacy marking is even **detectable** depends on the source tool — all flagged "verify your licensing/tenant" and proven in simulation, never assumed.

1. Solution Overview — Sensitivity Label Migration Project (Microsoft Purview)

What this solution is

Many organizations classified their content **before** Microsoft Purview — with an on-premises or third-party classification tool that stamped a marking on every document and email, or with an older Microsoft labeling client. Those legacy markings are **not** Microsoft Purview sensitivity labels: they don't drive Purview encryption, DLP, retention, or Copilot data protection, and they leave the estate in two classification schemes at once. Left unaddressed, that is both a **compliance gap** (governance keyed off labels that Purview doesn't understand) and a **data-risk exposure** (content the new controls can't see). This solution **maps** every legacy/third-party classification to a target Microsoft Purview **sensitivity label** and **reclassifies** the content onto those labels — with the reclassification **piloted in simulation first**, validated for precision, and only then enforced.

- **Inventory both sides:** the **legacy classification values** the source tool wrote (exported from that tool), and the **target Purview estate** — existing sensitivity labels, label policies, auto-labeling policies, and sensitive information types.
- Build a **mapping matrix**: every distinct legacy classification value → exactly one target Purview sensitivity label, with unmapped values flagged as explicit owner decisions.
- Build the **detection** for each legacy value — a **sensitive information type (SIT)** or content/metadata condition Purview can match on — and an **auto-labeling policy** that applies the mapped target label wherever that detection fires.
- **Pilot the reclassification in simulation** (the auto-labeling policy scans and reports **without labeling anything**), measure precision against the mapping, then** enforce from simulation** to reclassify.
- Validate coverage and precision against pass/fail gates, then **expand** location-by-location.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** — no third-party tooling on the target side.

> ***This is a migration onto an existing target taxonomy — not a taxonomy build.*** *It assumes the* ***destination*** *Microsoft Purview sensitivity labels already exist (or are being built by the separate labels-implementation solution). Designing the label taxonomy, authoring DLP, and standing up retention are separate Purview solutions; where this guide reaches their edge it points to them rather than duplicating them.*

Why this matters

• **Two classification schemes is a governance gap.** While content carries a legacy marking that Purview can't read, none of Purview's controls — encryption, DLP, retention, Copilot protection — apply to it. The point of the migration is to collapse to **one** scheme that the platform actually enforces.

• **Legacy markings don't convert themselves.** There is **no supported cmdlet that turns a third-party classification value into a Purview label. **Each legacy value must be *detected* and *reclassified* onto its mapped label. This solution treats the** mapping matrix **and its** detection precision** as the deliverables, not an afterthought.

• **Reclassification is high-impact.** Applying a Purview label — especially an **encrypting** one — to a large body of existing content changes how every one of those files is protected. Doing it in **simulation first**, measuring false positives, and enforcing only after sign-off is the difference between a clean migration and a tenant-wide incident (surprise encryption, broken co-authoring).

• **Detectability is the real risk, and simulation is how you retire it.** Whether Purview can reliably match a given legacy marking depends entirely on **what the source tool wrote and where** (document content, document metadata, or an email header) — this is verified empirically in simulation, not assumed.

How it works

1. **Assess.** Read-only: inventory the **legacy classification values** (from the source tool's own export) and the **target Purview estate** (labels, label policies, auto-labeling policies, custom SITs). No changes.

1. **Design.** Build the **mapping matrix** — every legacy value → one target sensitivity label — and, for each value, decide the **detection method** (a custom SIT matching the marking text/metadata, a keyword, or a trainable classifier). Flag any legacy value with no clean target as an explicit owner decision.

1. **Configure (detection + simulation).** Create/confirm the detection SITs, then create the **auto-labeling policies in simulation** (`TestWithoutNotifications`) that apply each mapped target label wherever its detection fires — scoped to the pilot locations. Preview every write with `-WhatIf` first.

1. **Pilot (measure precision).** Let simulation scan the in-scope locations and produce a match report. **Nothing is labeled yet.** Review the matches against the mapping with the data owner.

1. **Validate.** Confirm mapping **coverage** (every in-scope legacy value is detected and mapped) and **precision** (the simulated reclassification hits the right content) against explicit pass/fail gates.

1. **Expand (enforce = reclassify).** Only after precision sign-off, **enforce from simulation** so the target labels are actually applied, then broaden location-by-location and reconcile the residual legacy markings.

The migration mapping (what the matrix covers)

Mapping dimension	Legacy / third-party source	Purview target
Classification value	A marking string / metadata written by the source tool (e.g. a `Classification` document property, an email `x-header`, a footer marking)	Exactly one Microsoft Purview **sensitivity label**
Detection	How Purview *recognizes* the legacy value	A **custom SIT** (regex/keyword), content marking keyword, or trainable classifier
Protection	Any encryption/rights the legacy tool applied out-of-band	The target label's Purview encryption/rights (parity is a **records/security decision**, not automatic)
Reclassification	The legacy marking left in place	The mapped Purview label **applied by auto-labeling** (enforced from simulation)
Residual marking	The old visual/metadata marking still on the file	Reconciliation decision — leave, overwrite, or clean up (owner call)

> ***What the source tool wrote, and whether Purview can detect it, must be verified per tool.*** *Some tools write an extractable* ***content marking*** *(matchable by a keyword SIT); some write* ***document metadata / custom properties****; some write* ***email x-headers****. Detectability is* ***flagged and proven in simulation****, never assumed — this is the single most important honesty in the project.*

What you can control

• The **mapping matrix** — which legacy value maps to which target label, and which values are out of scope or need an owner decision.

• The **detection method** per legacy value (custom SIT text/regex, keyword, or trainable classifier).

• The **pilot locations** (which SharePoint sites / OneDrive accounts / mailboxes are reclassified first) and the change window per wave.

• Whether a mapped target label **encrypts** — and therefore the blast radius of enforcement.

• The **residual-marking reconciliation** decision once content carries the new Purview label.

Limitations (be honest with buyers)

• **Auto-labeling is licensing-gated.** Service-side auto-labeling for SharePoint/OneDrive/Exchange — the engine that does the reclassification — requires **Microsoft 365 E5** or the **E5 Compliance** add-on. Manual labels and publishing work at E3, but the automated reclassification does not — **verify your licensing/tenant.**

• **Detectability is not guaranteed.** If the source tool wrote its marking somewhere Purview can't index (an opaque metadata blob, a proprietary property, an image-only footer), a content/metadata SIT may not match it. This solution **surfaces** that in simulation; it cannot make an undetectable marking detectable.

• **No auto-convert exists.** There is no supported cmdlet that converts a legacy label into a Purview label. Reclassification is detection-driven, which is why precision must be measured before enforcing.

• **Protection parity is a decision, not a copy.** Whether the mapped Purview label should reproduce encryption the legacy tool applied is a **records/security determination** — the method is here; the determination is deferred.

• **This is not a taxonomy, DLP, or retention build.** The target labels are assumed to exist; those builds are separate Purview solutions.

Naming convention (used in every doc and script)

Artifacts this solution creates or captures follow `<Purpose> – Purview – <Stage>` and are reused verbatim across the docs, scripts, and portal.

Artifact	Name
Legacy classification values inventory (from source tool)	`Legacy Classification Inventory – Migration – Assess`
Legacy → Purview label mapping matrix	`Label Mapping Matrix – Design`
Detection SIT for a legacy value	`SIT – Legacy <Value> – Detect`
Reclassification auto-labeling policy (simulation)	`Reclassify Sim – <Label> – Locations`
Migration coverage & precision report	`Migration Coverage – Report`
Migration scope & mapping-health map	`Migration Scope – Map`
Reclassification pilot ring	`Label Migration Pilot Ring – Stage1`

The standard deployment model

> *Assess both sides → design the mapping matrix + detection → configure detection and auto-labeling **in simulation** → pilot to measure precision → validate coverage + precision against pass/fail gates → **enforce from simulation to reclassify** and expand location-by-location. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce).***

2. Requirements Guide — Sensitivity Label Migration Project

Verify **every** requirement below before starting the Implementation Guide. These are the **solution-specific** prerequisites for migrating legacy/third-party classifications onto Microsoft Purview sensitivity labels; they build on the shared **Global Requirements — Microsoft Purview Solutions** (compliance licensing, least-privilege roles, Security & Compliance PowerShell + Graph toolchain, in-scope workloads, compliance/records/legal stakeholders, and a pilot scope with change control). Read the Global Requirements first; this guide does not repeat them.

> ***Deferral note (records/security/legal).*** *Whether a mapped Purview label should* ***encrypt*** *or reproduce protection the legacy tool applied, and whether a legacy value maps to a "record," are* ***determinations owned by your records, security, compliance, and legal functions*** *— not technical defaults. This solution provides the* ***mapping method, detection, and evidence****; it does not decide the target protection. Confirm every mapping and any encryption with the accountable owner before enforcing.*

2.1 Licensing for auto-labeling (the reclassification engine)

• **Description:** The reclassification is performed by **service-side auto-labeling** for

SharePoint/OneDrive and Exchange, which requires **Microsoft 365 E5** or the **E5 Compliance** add-on. **Client-side** auto-labeling (labels applied/recommended in Office apps) and several advanced classifiers are also E5-tier. Manual sensitivity labels and publishing are available at E3, but the **automated migration** in this solution depends on auto-labeling.

• **Why required:** Without auto-labeling licensing there is no supported way to reclassify existing

content at scale onto the mapped labels.

• **How to verify:** M365 admin center → subscriptions/add-ons; confirm **Information Protection →

Auto-labeling **is visible and usable in the Purview portal.** Verify your licensing/tenant** — Microsoft's licensing tables are the authority.

• **How to implement if missing:** Assign/purchase E5 or E5 Compliance for the migration scope before

configuring auto-labeling. (Manual re-labeling by users is a fallback but does not scale for a migration.)

2.2 The target sensitivity labels already exist and are published

• **Description:** The **destination** Microsoft Purview sensitivity labels every legacy value will map

to must **already exist** in the tenant (created here or by the separate labels-implementation solution) and, for the pilot scope, be **published** so they can be applied. Each target label's protection (marking/encryption) must be the intended end state.

• **Why required:** Auto-labeling can only apply a label that exists and is available; the mapping

matrix has no valid target otherwise.

• **How to verify:** `Get-Label` / Purview → **Information Protection** → **Sensitivity labels** shows every

intended target label; `Get-LabelPolicy` shows it published to the pilot scope.

• **How to implement if missing:** Build/publish the target taxonomy first (see the sensitivity &

retention labels implementation solution). Do not invent target labels inside this migration.

2.3 A legacy classification export from the source tool

• **Description:** An export/report from the **source classification tool** (or a representative sample)

listing the **distinct classification values** in use and, where the tool provides it, **how each value is written** (a document property/metadata field, an email header, a footer marking string). This is the input to the mapping matrix.

• **Why required:** You cannot map or detect values you haven't enumerated; the detection method depends

on *where* the source tool wrote the marking.

• **How to verify:** Obtain the source tool's classification list / reporting; confirm it enumerates the

full set of values and, ideally, the metadata/marking each writes.

• **How to implement if missing:** Extract the value list from the source tool's admin console or a

content sample before authoring the mapping. Capture at least one **example file/email per value** for detection testing.

2.4 Detectability confirmed per legacy value

• **Description:** For each legacy value, a confirmed answer to **"can Purview detect this?"** — i.e.

whether the marking appears in **extractable content** (matchable by a keyword/regex custom SIT), in **document metadata Purview can index**, or in an **email x-header**, versus an opaque/proprietary store Purview cannot read.

• **Why required:** Detection precision is the whole migration risk. A value Purview cannot reliably

detect cannot be reclassified automatically and must be handled another way (manual, or source-tool remediation first).

• **How to verify:** For a sample file/email per value, confirm the marking string or metadata is

present in a way Purview can match; validate empirically in the **simulation** step. **Verify what your specific source tool writes** — common tools (e.g. Titus/Fortra, Boldon James, Janusseal, or a legacy Microsoft AIP client) differ, and the exact property/header names must be confirmed, never assumed.

• **How to implement if missing:** Where a value is undetectable, escalate: remediate at the source tool

(re-stamp to a detectable form), plan manual re-labeling for that subset, or scope it out with owner sign-off.

2.5 Information Protection role (least privilege)

• **Description:** The Purview role group this solution needs: **Information Protection Admin** (create/

manage sensitivity labels, label policies, auto-labeling policies) and **Compliance Administrator** or the appropriate role for **sensitive information types**. Global Administrator is **not** required for the migration work.

• **Why required:** The auto-labeling and SIT cmdlets (`New-AutoSensitivityLabelPolicy`,

`New-AutoSensitivityLabelRule`, `Set-AutoSensitivityLabelPolicy`, and SIT management) are gated by these role groups.

• **How to verify:** Purview portal → **Roles & scopes** → **Permissions**; confirm membership in *Information Protection* (and the SIT-management role).

• **How to implement if missing:** Add the admin to the least-privilege role group under change control; wait for propagation before running cmdlets.

2.6 Security & Compliance PowerShell + Graph connectivity

• **Description:** A workstation with the **ExchangeOnlineManagement** module (providing `Connect-IPPSSession` for Security & Compliance PowerShell) and, where Graph is used for licensing/evidence, **Microsoft Graph PowerShell** — with a modern-auth / MFA-capable admin account.

• **Why required:** All five provided scripts and every configuration cmdlet run through `Connect-IPPSSession`.

• **How to verify:** `Connect-IPPSSession` succeeds and `Get-Label` / `Get-AutoSensitivityLabelPolicy` return without error.

• **How to implement if missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; confirm modern-auth sign-in. See the Global Requirements for the full toolchain.

2.7 A pilot scope and a simulation-first commitment

• **Description:** A named **pilot scope** — a limited set of SharePoint sites / OneDrive accounts / mailboxes to reclassify first — and an explicit commitment that **every reclassification auto-labeling policy is created in *simulation* mode** and its precision validated before it is enforced. Encrypting target labels are piloted before any broad enforcement.

• **Why required:** Reclassifying existing content (and especially applying encryption to it) is high-impact and hard to reverse cleanly; the house model requires a simulate/measure pass first.

• **How to verify:** Confirm the pilot locations, a change window, and a named approver; confirm the target labels are published to the pilot scope (§2.2).

• **How to implement if missing:** Define the pilot locations and agree the change-control path before configuring any auto-labeling.

2.8 A residual-marking reconciliation decision

• **Description:** A decision, per mapping, on what happens to the **old legacy marking** once the Purview label is applied — leave it in place, let the Purview label's own marking co-exist/overwrite, or plan a source-tool cleanup. And the plan for **decommissioning the source tool** once migration is complete.

• **Why required:** Without this, content ends up double-marked indefinitely, which confuses users and undermines the "one scheme" goal of the migration.

• **How to verify:** Confirm the reconciliation approach with the content/records owner and the source tool's admin; confirm whether the Purview label's content marking should replace the legacy footer.

• **How to implement if missing:** Agree the reconciliation and source-tool decommission plan before broad enforcement; capture it in the mapping matrix.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Sensitivity Label Migration Project

Follow these steps **in order**. Work is done in the **Microsoft Purview portal** (purview.microsoft.com), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph**. Every write is previewed with `-WhatIf` (the default) before `-Commit`, and every reclassification auto-labeling policy is created **in simulation** before it is enforced. Artifact names follow the `<Purpose> – Purview – <Stage>` convention from the Overview and are reused verbatim.

> ***Model:** Assess both sides → design the mapping matrix + detection → configure detection and auto-labeling **in simulation** → pilot to measure precision → validate coverage + precision → **enforce from simulation to reclassify** and expand location-by-location. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce). Auto-labeling is E5-gated and the reclassification cannot be undone by a single "unlabel" button — pilot in simulation and measure precision before enforcing.** The target labels must already exist and be published (Requirement 2.2). Portal paths are current at authoring — the Purview portal evolves, so **verify paths in your tenant.***

Step 1 — Inventory both sides (Assess)

1. Export the **legacy classification values** from the source tool (Requirement 2.3) to a CSV — one row

per distinct value, with (where available) the marking string, the metadata property or email header it writes, and an example item. Save it as `Legacy Classification Inventory – Migration – Assess`.

1. Run `IPMIG-Discovery.ps1 -LegacyCsv .\LegacyValues.csv -CsvFolder .\out`. Read-only, it captures the

target Purview estate — sensitivity labels (`Get-Label`), label policies (`Get-LabelPolicy`), existing auto-labeling policies + mode (`Get-AutoSensitivityLabelPolicy`), and custom sensitive information types (`Get-DlpSensitiveInformationType`) — and cross-references your legacy CSV, listing each distinct legacy value and whether a matching target label / detection SIT already exists.

1. In the portal, open **Information Protection** → **Sensitivity labels**, **Label policies**, and **Auto-labeling** (verify paths) and confirm the on-screen estate matches the inventory.

1. **Pass/fail:** you have a dated inventory of every distinct legacy value **and** the full target estate — the baseline both sides of the migration map onto.

Step 2 — Build the mapping matrix and choose detection (Design)

1. Build `Label Mapping Matrix – Design` — one row per legacy value:

• **Target label** → exactly one existing, published Purview sensitivity label (confirm it exists, Requirement 2.2). If two legacy values should collapse to one label, that is allowed; if a value has **no clean target**, flag it as an explicit **owner decision** (do not guess).

• **Detection method** → how Purview will recognize the value: a **custom SIT** matching the marking text/metadata, a **keyword**, or a **trainable classifier**. Record the exact match string (Requirement 2.4) and the SIT name `SIT – Legacy <Value> – Detect`.

• **Protection intent** → whether the target label encrypts, and whether that reproduces protection the legacy tool applied (a **records/security decision**, deferred — Requirement 2.1 note).

• **Locations in scope** and the **residual-marking reconciliation** decision (Requirement 2.8).

1. Author the **detection SITs** in the Purview portal (**Data classification → Sensitive info types → Create**) as a keyword/regex that matches each legacy marking. The portal is the documented path for custom-SIT authoring; name each `SIT – Legacy <Value> – Detect`. (The provided scripts reference** these SITs by name; confirm they exist with `IPMIG-Discovery.ps1` before Step 3.)

1. **Pass/fail:** every in-scope legacy value has (a) one target label, (b) a named detection SIT that exists, and (c) a reconciliation decision. Unmapped values are named and owned, not silently dropped.

Step 3 — Configure reclassification in simulation (Configure, preview-first)

1. Put the matrix into a mapping CSV (`LegacyValue,TargetLabel,SitName,Locations`) and dry-run:

`IPMIG-Set-Migration.ps1 -MappingCsv .\mapping.csv -PilotSites <sites> -WhatIf`. With `-WhatIf` (the default) it makes **no changes** — it prints exactly which auto-labeling policy + rule it would create per mapping (`New-AutoSensitivityLabelPolicy -Mode TestWithoutNotifications`, `New-AutoSensitivityLabelRule -ContentContainsSensitiveInformation <SIT>` applying the mapped label) and confirms each referenced SIT and target label exists.

1. Read every line against `Label Mapping Matrix – Design`. Confirm each policy targets the **pilot locations only**, applies the correct **target label**, and is in simulation (`TestWithoutNotifications`) — it will not** label anything yet.

1. Apply to the pilot: `IPMIG-Set-Migration.ps1 -MappingCsv .\mapping.csv -PilotSites <sites> -Commit`.

This creates `Reclassify Sim – <Label> – Locations` policies in **simulation**. Nothing is reclassified.

1. **Pass/fail + rollback:** `Get-AutoSensitivityLabelPolicy` shows each policy in a `Test*`

(simulation) mode, scoped to the pilot locations, applying the mapped label. Rollback: `IPMIG-Set-Migration.ps1 -PolicyName '<name>' -RemovePolicy -Commit` deletes the simulation policy (no content was touched).

Step 4 — Run simulation and measure precision (Pilot)

1. Let the simulation scan the in-scope pilot locations (service-side scanning processes volume over

time — **verify current scan limits for your tenant**). It scans and reports; **no content is labeled.**

1. Run `IPMIG-Validation.ps1 -SimReport -PolicyName 'Reclassify Sim – <Label> – Locations'` to report

each policy's simulation mode/progress and the **items it would reclassify**.

1. Review the match set with the **data owner** against the mapping: sample matched items for **false

positives **(content that would get the wrong label) and estimate** false negatives** (legacy-marked content the detection missed — Requirement 2.4).

1. **Pass/fail:** simulation has scanned, produced a match report, and the false-positive rate is within

the agreed threshold. **This is the gate before enforcement.** If precision is poor, tune the detection SIT (tighter regex, higher instance counts, supporting keywords) in the portal and re-run simulation — do **not** enforce.

Step 5 — Validate coverage and precision (Validate)

1. Run `IPMIG-Report.ps1 -MappingCsv .\mapping.csv -CsvPath .\MigrationCoverage.csv` to produce

`Migration Coverage – Report`: every legacy value → mapped/unmapped, detection SIT present, policy in simulation vs enforced, and the simulated match count per policy.

1. Run `IPMIG-Scope-Health.ps1 -MappingCsv .\mapping.csv` for the **mapping-health** check — flags any

legacy value with **no mapping**, a mapping referencing a **missing** SIT or target label, or a policy **enforcing before sign-off**.

1. **Pass/fail:** every in-scope legacy value is mapped and detected; no mapping references a missing

object; every reclassification policy is still in simulation and has acceptable precision. Do **not** expand with an unmapped in-scope value or an unresolved detection gap.

Step 6 — Enforce from simulation to reclassify (Expand — pilot locations)

1. Only after precision sign-off (Step 4/5), enforce the pilot policies:

`IPMIG-Set-Migration.ps1 -PolicyName 'Reclassify Sim – <Label> – Locations' -EnforceFromSimulation -Commit`. This flips the accepted simulation policy to `Enable`, so the

mapped target labels are now **actually applied** to the matching content — the reclassification.

1. Re-run `IPMIG-Validation.ps1 -SimReport -PolicyName '<name>'` and spot-check newly labeled items:

the mapped label is applied, and any **encryption/marking** on the target label behaves as designed (co-authoring, access) — for encrypting labels, confirm an unauthorized user is denied.

1. Apply the **residual-marking reconciliation** decision (Requirement 2.8) for the pilot content.

2. **Pass/fail + rollback:** pilot content now carries the mapped Purview label; no surprise encryption

or broken access. Rollback: return the policy to simulation (`IPMIG-Set-Migration.ps1 -PolicyName '<name>' -ReturnToSimulation -Commit`) — this stops further labeling; already-applied labels are reverted per your label-removal process (document it).

Step 7 — Expand location-by-location and decommission the source tool (Expand)

1. With the pilot validated, widen each reclassification policy's **locations** to the next wave

(`IPMIG-Set-Migration.ps1 -PolicyName '<name>' -AddSites <sites> -WhatIf` then `-Commit`), repeating the simulation → measure → enforce pattern per wave where a new location introduces new content patterns. Re-run `IPMIG-Report.ps1` after each wave.

1. Reconcile residual legacy markings per wave, and — once a population is fully reclassified and

validated — **retire the source classification tool** for it per your decommission plan (Requirement 2.8). Keep the source tool's export as migration evidence.

1. **Pass/fail:** the full in-scope estate carries mapped Purview labels, `Migration Coverage – Report`

shows no unmapped in-scope value and no enforcing policy with poor precision, and the source tool is decommissioned on schedule.

Order of operations

`Inventory legacy values + target estate → build mapping matrix + author detection SITs → dry-run (-WhatIf) reclassification policies → create them in SIMULATION (pilot locations) → run simulation + measure precision with the data owner → validate coverage + precision → enforce from simulation to reclassify the pilot → expand location-by-location + reconcile markings + decommission the source tool`

5. Testing & Validation Guide — Sensitivity Label Migration Project

Run these tests after the Implementation Guide, **in order**. Each has an explicit **pass/fail** signal and a rollback. Do not enforce a reclassification policy, and do not expand from the pilot locations, until the relevant tests pass. All checks use the Purview portal and Security & Compliance PowerShell — the provided scripts automate the read-only ones.

> ***Reminder:*** *whether a mapped label should* ***encrypt****, and whether a legacy value is a "record," are records/security/legal determinations from §2 — you are validating that the* ***mapping and reclassification behave as specified****, not deciding the target protection.*

Test 1 — Both sides inventoried and cross-referenced

- **How:** `IPMIG-Discovery.ps1 -LegacyCsv .\LegacyValues.csv -CsvFolder .\out`.
- **Pass:** every distinct legacy value from the source export is listed, and for each the report shows whether a target label and a detection SIT already exist — the full baseline.
- **Fail:** the legacy CSV is incomplete, or target labels are missing (build/publish them first, §2.2) → fix the inputs and re-run.

Test 2 — Mapping matrix is complete

- **How:** review `Label Mapping Matrix – Design` with `IPMIG-Scope-Health.ps1 -MappingCsv .\mapping.csv`.
- **Pass:** every **in-scope** legacy value maps to exactly one existing, published target label and names a detection SIT; every unmapped value is flagged as an explicit owner decision.
- **Fail:** any in-scope value with no target, a mapping to a non-existent label, or a duplicate/ambiguous mapping → resolve with the owner before configuring.

Test 3 — Detection SITs exist and match a sample

- **How:** `Get-DlpSensitiveInformationType | Where-Object Name -like 'SIT - Legacy*'`, then test each SIT against a **sample legacy-marked item** (portal test pane, or apply the SIT in a test).
- **Pass:** every `SIT – Legacy <Value> – Detect` exists and fires on a genuinely legacy-marked sample, and does **not** fire on clearly unmarked content.
- **Fail:** a SIT is missing, or does not detect the legacy marking (the value may be in an opaque store, §2.4) → re-author the SIT, or escalate the value as undetectable (manual/source remediation).

Test 4 — Reclassification policies are created in simulation only

- **How:** `IPMIG-Validation.ps1 -Full`, or `Get-AutoSensitivityLabelPolicy | Format-Table Name,Mode`.
- **Pass:** every `Reclassify Sim – <Label> – Locations` policy is in a **simulation** mode (`TestWithoutNotifications`), scoped to the **pilot locations**, applying the **mapped** label — and **nothing has been labeled**.
- **Fail:** a policy is enforcing before sign-off, or scoped tenant-wide → return it to simulation

(`-ReturnToSimulation`) / re-scope before proceeding.

Test 5 — Simulation has scanned and produced a match report

- **How:** `IPMIG-Validation.ps1 -SimReport -PolicyName 'Reclassify Sim – <Label> – Locations'`.
- **Pass:** the policy has scanned the in-scope locations and produced a match set — the **preview** of what it would reclassify — with **no content actually labeled**.
- **Fail:** no results yet (allow scan time; confirm the locations contain matching content) or the policy is not in simulation → investigate before measuring precision.

Test 6 — Reclassification precision is acceptable (the enforcement gate)

- **How:** review the `IPMIG-Validation.ps1 -SimReport` match set with the **data owner**; sample matched items for false positives and estimate false negatives against the legacy inventory.
- **Pass:** the false-positive rate is within the agreed threshold, coverage of known legacy-marked content is acceptable, and the data owner **accepts the match set** for enforcement.
- **Fail:** too many wrong-label matches, or too much legacy content missed → tune the detection SIT (tighter regex, higher instance counts, keywords/classifier) in the portal, re-run simulation, re-review. **This is the gate before any enforcement.**

Test 7 — Enforced reclassification matches the accepted simulation

- **How:** after `-EnforceFromSimulation` (Step 6), re-run `IPMIG-Validation.ps1 -SimReport` and spot-check newly labeled items.
- **Pass:** the mapped target label is applied to the same content simulation predicted; nothing outside the accepted match set was labeled.
- **Fail:** divergence or unexpected labeling → return the policy to simulation (`-ReturnToSimulation`) and investigate before continuing.

Test 8 — Encryption/marking on a mapped label behaves as designed

- **How:** for any mapped label that **encrypts**, take a reclassified test item and attempt to open it as a user **outside** the label's rights; confirm content marking (header/footer/watermark) appears.
- **Pass:** the unauthorized user is **denied**; authorized rights behave as defined; marking appears where configured — the reclassification's protection is correct.
- **Fail:** unauthorized access succeeds, co-authoring breaks unexpectedly, or marking is wrong → the

target label's protection is misconfigured. **Halt broad enforcement** of encrypting mappings until fixed (this is why encryption is piloted).

Test 9 — Residual legacy marking is reconciled as decided

- **How:** inspect reclassified pilot content for the **old** legacy marking and confirm it matches the §2.8 reconciliation decision (left, co-existing, or cleaned up).
- **Pass:** content carries the intended Purview label and the residual legacy marking is in the agreed end state — no undocumented double-marking.
- **Fail:** unexpected double-marking or a lost marking → apply the reconciliation step / revisit the decision with the content owner.

Test 10 — Coverage & mapping-health check before expansion

- **How:** `IPMIG-Report.ps1 -MappingCsv .\mapping.csv -CsvPath .\MigrationCoverage.csv` and `IPMIG-Scope-Health.ps1 -MappingCsv .\mapping.csv`.
- **Pass:** every in-scope legacy value is mapped + detected; no mapping references a missing SIT/label; every policy is either still in simulation or enforcing **with signed-off precision** — all green.
- **Fail:** any unmapped in-scope value, missing object, or a policy enforcing without precision sign-off → resolve before widening locations.

Exit criteria (all must be true before broad enforcement)

`Both sides inventoried · mapping matrix complete · detection SITs exist and match · policies created in simulation only · simulation scanned + match report produced · precision accepted by the data owner · enforced == accepted simulation · encryption/marking correct · residual markings reconciled · coverage + health check green · rollback (return-to-simulation / remove-policy) documented per policy`

6. Troubleshooting — Sensitivity Label Migration Project (Microsoft Purview)

Common issues, grouped by step. Each has a **symptom**, the **likely cause**, and a **resolution**. Every write in this solution is `-WhatIf` by default and every reclassification policy is created **in simulation** first — `IPMIG-Discovery.ps1`, `IPMIG-Report.ps1`, `IPMIG-Scope-Health.ps1`, and `IPMIG-Validation.ps1` are read-only; only `IPMIG-Set-Migration.ps1` writes, and it labels no content until you enforce from simulation after precision sign-off. **Auto-labeling is E5-gated and reclassification is not undone by one button — pilot in simulation and measure before enforcing.**

Licensing & connectivity

Symptom: `Get-AutoSensitivityLabelPolicy` / `New-AutoSensitivityLabelPolicy` not found or errors. **Likely cause:** service-side auto-labeling is licensing-gated (typically E5), or `ExchangeOnlineManagement` isn't connected. **Resolution:** `Install-Module ExchangeOnlineManagement`, `Connect-IPPSSession`, and **verify your licensing/tenant** for service-side auto-labeling before Step 3. If unlicensed, that is a prerequisite gap, not a script bug.

Inventory & mapping (Steps 1–2)

Symptom: `IPMIG-Discovery.ps1` shows a legacy value with no matching target label or SIT. **Likely cause:** the target taxonomy isn't published yet, or the detection SIT hasn't been authored. **Resolution:** publish the target label (Requirement 2.2, its own taxonomy solution) and author the detection SIT in the portal (`SIT – Legacy <Value> – Detect`) before Step 3. Confirm with `IPMIG-Discovery.ps1`.

Symptom: a legacy value has no clean target label. **Likely cause:** this is a mapping **owner decision**, not a technical gap. **Resolution:** flag it explicitly in `Label Mapping Matrix – Design` and route it to the data owner — do **not** guess a mapping. `IPMIG-Scope-Health.ps1` will keep flagging it until resolved.

Creating simulation policies (Step 3)

Symptom: `IPMIG-Set-Migration.ps1` skips a mapping row with "target label not found" / "detection SIT not found". **Likely cause:** the CSV references a label or SIT that doesn't exist (name mismatch or not yet created). **Resolution:** fix the `mapping.csv` name to match exactly, or create/publish the object first. The guard is intentional — it won't build a policy on a missing object.

Symptom: nothing was created. **Likely cause:** `-Commit` was omitted (default is `-WhatIf`). **Resolution:** review the previewed `New-AutoSensitivityLabelPolicy/Rule` lines, then re-run with `-Commit`. Policies are created in **simulation** (`TestWithoutNotifications`) only.

Running simulation & measuring precision (Step 4)

Symptom: simulation shows few/no matches after creation. **Likely cause:** service-side scanning processes volume over time; the scan hasn't completed, or the detection SIT is too tight. **Resolution:** allow time (verify current scan limits for your tenant) and re-check with `IPMIG-Validation.ps1 -SimReport`. If the SIT truly misses legacy-marked content (false negatives), loosen the regex / lower instance count in the portal and re-simulate — do **not** enforce to compensate.

Symptom: the per-item match set isn't in the script output. **Likely cause:** the detailed simulated match set lives in the portal, not a public cmdlet. **Resolution:** open **Information Protection** → **Auto-labeling** → **[policy]** → **Simulation** to sample matches for false positives with the data owner. `IPMIG-Validation.ps1` reports mode/status/config and points you there.

Symptom: false-positive rate is too high. **Likely cause:** the detection SIT is too broad (matches look-alike content). **Resolution:** tune the SIT (tighter regex, higher instance count, supporting keywords) in the portal and re-simulate. Precision sign-off by the data owner is the **gate before enforcement**.

Coverage & health (Step 5)

Symptom: `IPMIG-Report.ps1` shows `LabelExists=MISSING` though the label seems to exist. **Likely cause:** name mismatch (DisplayName vs Name), or the label exists but isn't **published**. **Resolution:** match the exact label name in the CSV; confirm the label is **published** in **Label policies** (published-scope

is portal-authoritative — the report notes this).

Symptom: `IPMIG-Scope-Health.ps1` flags a policy "ENFORCING before sign-off". **Likely cause:** a reclassification policy is in `Enable` mode without recorded precision sign-off. **Resolution:** confirm the data owner signed off (Step 4/5); once recorded, re-run with `-AllowEnforced`, or return the policy to simulation (`IPMIG-Set-Migration.ps1 -PolicyName '<name>' -ReturnToSimulation -Commit`).

Enforcing & rollback (Steps 6–7)

Symptom: after enforcing, users hit unexpected encryption / access denial. **Likely cause:** the target label applies encryption that the legacy tool did not — a protection-intent difference (Requirement 2.1 note). **Resolution:** confirm the protection intent per label was reviewed as a records/security decision before enforcing. Rollback: `IPMIG-Set-Migration.ps1 -PolicyName '<name>' -ReturnToSimulation -Commit` stops further labeling; already-applied labels are reverted per your documented label-removal process.

Symptom: you need to widen locations. **Resolution:** `IPMIG-Set-Migration.ps1 -PolicyName '<name>' -AddSites <sites> -WhatIf` then `-Commit`; re-simulate for a wave whose new location introduces new content patterns before enforcing it.

What this solution does NOT decide

Whether a target label's **protection intent** correctly reproduces the legacy tool's protection, the **acceptable false-positive threshold**, and **when to decommission** the source tool are **records / security / data-owner determinations** (Requirement 2.1, 2.8). This solution delivers the mapping, detection, simulation evidence, and the enforced reclassification — not the determination.

7. Best Practices — Sensitivity Label Migration Project

Field-tested practices for migrating a legacy/third-party classification estate onto Microsoft Purview sensitivity labels **without mislabeling content or surprising users**. These complement the Implementation and Testing guides.

Mapping design

- **One target label per legacy value.** Keep the matrix unambiguous — each legacy value resolves to exactly one Purview label. Collapsing several legacy values to one label is fine; a value with two possible targets is a decision to make **before** configuring, not a coin flip at enforcement.
- **Map to an existing, published label — never invent a target.** The destination taxonomy is a separate build. If a legacy value has no clean home, that is a finding for the label owner (extend the taxonomy, or scope the value out), not a new label improvised inside the migration.
- **Name every unmapped value as an owner decision.** Silent gaps are how legacy content stays ungoverned. The matrix must show *why* a value is unmapped and *who* owns the decision.
- **Record the protection intent explicitly.** Whether the mapped label encrypts — and whether that

reproduces protection the legacy tool applied — is a records/security determination. Capture it in the matrix; don't let encryption "come along for free" unexamined.

Detection (the real risk)

- **Prove detectability per value before you trust it.** What the source tool wrote — extractable content, indexable metadata, an email header, or an opaque blob — determines whether Purview can match it. Confirm on a sample per value; verify the exact property/header names for your specific source tool, never assume.
- **Prefer specific detections over broad ones.** A tight regex/keyword that matches the exact legacy marking string beats a loose one that also catches unrelated content. False positives here mean **mislabeled** content.
- **Escalate the undetectable, don't fudge it.** If Purview can't reliably see a marking, re-stamp it at the source tool into a detectable form, plan manual re-labeling for that subset, or scope it out with sign-off — never enforce a detection you know is unreliable.

Simulation & precision

- **Always simulate the reclassification first.** Never create a reclassification auto-labeling policy in Enable mode. Simulation (`TestWithoutNotifications`) scans and reports **without labeling** — it is the single most important safety control in this migration.
- **Measure precision with the data owner, then enforce.** Sample simulation matches for false positives and estimate false negatives against the legacy inventory. Enforcing a noisy detection reclassifies content wrongly — and if the label encrypts, that's surprise encryption at scale.
- **Watch scan latency and limits.** Service-side scanning processes large volumes over time and is subject to throughput/item limits — **verify current limits for your tenant** and set expectations; "no matches yet" is often "still scanning," not "nothing to find."

Enforcement & protection

- **Enforce encrypting mappings last and smallest.** Applying an encrypting label to existing content is the highest-blast-radius action in the project. Pilot it on a tiny, disposable-safe set first, prove the unauthorized-open test, and only then widen.
- **Expand location-by-location.** Widen each reclassification policy's scope in waves, re-simulating where a new location introduces new content patterns. A tenant-wide flip skips the cheap place to catch a mapping error.
- **Keep a return-to-simulation rollback per policy.** Flipping a policy back to simulation stops further labeling immediately; document how already-applied labels are removed for each mapping before you enforce.

Reconciliation & decommission

- **Decide the residual-marking end state up front.** Content that carries both the old legacy footer and the new Purview label confuses users. Agree — leave, co-exist, or clean up — before enforcement, per mapping.
- **Decommission the source tool on a schedule, not all at once.** Retire the legacy classifier for a population only after it is fully reclassified and validated; keep the source export as migration evidence.
- **Reclassify to one scheme, then govern from it.** The migration's payoff is that Purview controls — DLP, retention, Copilot protection — now key off labels the platform understands. Point those controls at the migrated labels once coverage is proven.

Program & change management

- **Pilot locations, always.** Reclassify a small set of sites/mailboxes before the estate; the pilot surfaces detection, marking, and encryption issues cheaply.
- **Communicate before broad reclassification.** Tell users which markings are changing to which labels and what they'll see (new marking/encryption), and where to get help — the ticket flood comes from surprise.
- **Keep least-privilege roles.** Information Protection Admin (plus SIT management) is enough; don't grant Global Admin for migration work.
- **Re-baseline periodically.** Re-run `IPMIG-Discovery.ps1` / `IPMIG-Report.ps1` to catch new legacy values appearing (a source tool still in use) and to track coverage over the migration's life.

Security & safety hygiene

- **No credentials in code.** All scripts authenticate interactively via `Connect-IPPSSession` / `Connect-MgGraph`; never embed secrets. Store exports securely — a legacy classification inventory can itself be sensitive.
- **Read-only or `-WhatIf` by default.** Every provided script defaults to no-change; writes require an explicit `-Commit`, and enforcement requires an explicit `-EnforceFromSimulation`. Keep it that way.
- **Verify licensing/tenant before relying on auto-labeling.** Service-side auto-labeling is E5-gated and the portal evolves — flag, don't assume.

8. Deliverables Summary — Sensitivity Label Migration Project

What you receive

A complete, self-guided package to **map** legacy/third-party classifications to Microsoft Purview sensitivity labels and **reclassify** the content onto them — detection, mapping matrix, auto-labeling in simulation, precision measurement, and enforcement — using only the Microsoft Purview portal and Security & Compliance PowerShell.

Documentation

Document	What it gives you
Solution Overview	The migration model (legacy value → mapped Purview label → detection → reclassify), the mapping dimensions, naming convention, honest limits (detectability, E5 gating, no auto-convert), and the deployment model.
Requirements Guide	Solution-specific prerequisites (auto-labeling licensing, target labels already published, a legacy export from the source tool, per-value detectability, Information Protection role, S&C PowerShell/Graph, a pilot scope + simulation-first commitment, and a residual-marking reconciliation decision) on top of the Purview Global Requirements.
Implementation Guide	Ordered steps: inventory both sides → build the mapping matrix + author detection SITs → create reclassification auto-labeling **in simulation** (`-WhatIf`→commit) → run simulation + measure precision → validate coverage → **enforce from simulation to reclassify** the pilot → expand location-by-location + reconcile markings + decommission the source tool. Each step named, gated, and reversible.
Testing & Validation Guide	10 pass/fail tests — inventory, mapping completeness, SIT detection, simulation-only posture, match report, precision sign-off (the enforcement gate), enforced==simulation, encryption/marking behavior, residual-marking reconciliation, and coverage/health.
Best Practices	Mapping, detection, simulation/precision, enforcement/protection, reconciliation/decommission, program/change-management, and safety guidance.
Troubleshooting Guide	The common failure modes (SIT doesn't detect, simulation stuck/empty, wrong-label matches, surprise encryption, double-marking, policy enforcing early) with resolutions.

PowerShell tools (5)

All run through `Connect-IPPSSession` (Security & Compliance PowerShell); read-only or `-WhatIf` by default, writes gated behind `-Commit` and enforcement behind an explicit `-EnforceFromSimulation`. No credentials in code.

Script	Role	Default
`IPMIG-Discovery.ps1`	**Two-sided inventory** — the target Purview estate (labels, label policies, auto-labeling policies + mode, custom SITs) cross-referenced against the legacy classification values exported from the source tool.	Read-only
`IPMIG-Set-Migration.ps1`	**Reclassification config/remediation helper** — from the mapping CSV, create the reclassification auto-labeling policies + rules **in simulation** (SIT → mapped label), then enforce-from-simulation, return-to-simulation, add locations, or remove a policy.	`-WhatIf`
`IPMIG-Validation.ps1`	**Simulation & health helper** — report a reclassification policy's simulation mode/progress and the items it would reclassify (`-SimReport`), and run a full label/policy/auto-label health check (`-Full`).	Read-only
`IPMIG-Report.ps1`	**Coverage & precision report** — every legacy value → mapped/unmapped, detection SIT present, policy simulation vs enforced, and simulated match count; CSV export.	Read-only
`IPMIG-Scope-Health.ps1`	**Mapping-health check** — flag legacy values with no mapping, mappings referencing a missing SIT or target label, and policies enforcing before sign-off.	Read-only

Outcomes

• A signed-off **mapping matrix** — every legacy/third-party classification value mapped to one existing Microsoft Purview sensitivity label.

• **Detection** proven per value in **simulation**, with precision measured and accepted before any content is touched.

• Content **reclassified** onto the mapped Purview labels by enforced auto-labeling — one classification scheme the platform actually governs.

• Residual legacy markings **reconciled** and the source tool **decommissioned** on a controlled schedule.

• Repeatable read-only tooling to re-baseline coverage and catch new legacy values over the migration's life.

Explicitly out of scope (deferred)

• **Target-label protection determinations** — whether a mapped label encrypts or reproduces the legacy tool's protection is owned by records/security/compliance/legal; this solution provides the mapping method and evidence, not the determination.

• **Building the destination taxonomy, DLP, and retention** — the target labels are assumed to exist (see the sensitivity & retention labels implementation and other Purview solutions).

• Any capability your **licensing/tenant** does not include — service-side auto-labeling (the reclassification engine) is E5-gated; verify before relying on it.

• **Making an undetectable legacy marking detectable** — where the source tool wrote to an opaque store, the fix is source-tool remediation or manual re-labeling, not an invented detection.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPMIG-Discovery.ps1

IPMIG-Report.ps1

Purpose: From the mapping CSV and the live Purview estate, produces the migration coverage report: for every legacy value — is it mapped to a target label, does its detection SIT exist, is the target label published, and is the reclassification policy in simulation vs enforced (with the rule's SIT). This is the coverage/status view used to gate expansion. Read-only — reads the estate and the CSV; writes only the local report CSV. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IPMIG-Report.ps1 -MappingCsv .\mapping.csv -CsvPath .\MigrationCoverage.csv Requires ExchangeOnlineManagement (Connect-IPPSSession). Mapping CSV columns: LegacyValue, TargetLabel, SitName (optional Locations). PowerShell 5.1+ or 7+. EXPECTED RESULTS A per-legacy-value table + optional CSV: Mapped (Y/N), SitExists, LabelPublished, PolicyMode (simulation/enforced/none), SimMatchNote. Any "N" / "missing" is a coverage gap to resolve before expanding. .NOTES Purview — Sensitivity Label Migration. Read-only. Reports coverage/state; whether precision is acceptable and whether protection intent is reproduced are data-owner/records determinations, not decided here. > [CmdletBinding()] param([Parameter(Mandatory)][string]$MappingCsv, [string]$CsvPath, [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\IPMIG-Report.ps1 -MappingCsv .\mapping.csv -CsvPath .\MigrationCoverage.csv Requires ExchangeOnlineManagement (Connect-IPPSSession). Mapping CSV columns: LegacyValue, TargetLabel, SitName (optional Locations). PowerShell 5.1+ or 7+. EXPECTED RESULTS A per-legacy-value table + optional CSV: Mapped (Y/N), SitExists, LabelPublished, PolicyMode (simulation/enforced/none), SimMatchNote. Any "N" / "missing" is a coverage gap to resolve before expanding. .NOTES Purview — Sensitivity Label Migration. Read-only. Reports coverage/state; whether precision is acceptable and whether protection intent is reproduced are data-owner/records determinations, not decided here. > [CmdletBinding()] param([Parameter(Mandatory)][string]$MappingCsv, [string]$CsvPath, [string]$UserPrincipalName)

IPMIG-Scope-Health.ps1

Purpose: Guardrail check before expansion. Reads the mapping CSV and the live estate and flags: any legacy value with NO mapping, any mapping referencing a detection SIT or target label that DOES NOT EXIST, and any reclassification policy already ENFORCING (Mode=Enable) — which must not happen before the data owner signs off precision. Read-only: makes no changes; it only reports health so you don't expand on a broken mapping. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IPMIG-Scope-Health.ps1 -MappingCsv .\mapping.csv Requires ExchangeOnlineManagement (Connect-IPPSSession). Mapping CSV columns: LegacyValue, TargetLabel, SitName (optional Locations). PowerShell 5.1+ or 7+. EXPECTED RESULTS A HEALTHY / issues-found summary. HEALTHY = every in-scope legacy value is mapped, every referenced SIT and target label exists, and no reclassification policy is enforcing prematurely. Any issue blocks expansion until resolved. .NOTES Purview — Sensitivity Label Migration. Read-only. An unmapped value flagged as an OWNER DECISION is a determination for the data owner, not a technical fix — surface it, don't guess a mapping. > [CmdletBinding()] param([Parameter(Mandatory)][string]$MappingCsv, [switch]$AllowEnforced, # suppress the "enforcing before sign-off" flag once sign-off is recorded [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\IPMIG-Scope-Health.ps1 -MappingCsv .\mapping.csv Requires ExchangeOnlineManagement (Connect-IPPSSession). Mapping CSV columns: LegacyValue, TargetLabel, SitName (optional Locations). PowerShell 5.1+ or 7+. EXPECTED RESULTS A HEALTHY / issues-found summary. HEALTHY = every in-scope legacy value is mapped, every referenced SIT and target label exists, and no reclassification policy is enforcing prematurely. Any issue blocks expansion until resolved. .NOTES Purview — Sensitivity Label Migration. Read-only. An unmapped value flagged as an OWNER DECISION is a determination for the data owner, not a technical fix — surface it, don't guess a mapping. > [CmdletBinding()] param([Parameter(Mandatory)][string]$MappingCsv, [switch]$AllowEnforced, # suppress the "enforcing before sign-off" flag once sign-off is recorded [string]$UserPrincipalName)

IPMIG-Set-Migration.ps1

IPMIG-Validation.ps1

Purpose: Validation counterpart to IPMIG-Set-Migration.ps1. For a named reclassification auto-labeling policy (or all "Reclassify Sim -*" policies) it reports the current mode (simulation vs enforced), the distribution/deployment status, the target label it applies, and the detection SIT its rule references — so you can confirm before enforcement that the policy is still in SIMULATION, scoped correctly, and applying the mapped label. Read-only: it makes no changes and labels no content. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IPMIG-Validation.ps1 -SimReport -PolicyName "Reclassify Sim - Confidential - Locations" powershell -ExecutionPolicy Bypass -File .\IPMIG-Validation.ps1 -SimReport # all reclassification policies Requires ExchangeOnlineManagement (Connect-IPPSSession). Role: Information Protection reader/admin. The detailed simulated match SET (per item) is shown in the Purview portal Auto-labeling > Simulation tab — this tool reports mode/status/config and points you there; it does not read item bodies. EXPECTED RESULTS Per policy: Mode (Test* = simulation PASS before enforcement), status, target label, rule SIT. If a policy shows Mode=Enable it is ENFORCING (reclassifying) — expected only after precision sign-off. .NOTES Purview — Sensitivity Label Migration. Read-only. Whether precision is acceptable and whether to enforce are data-owner/records decisions — this tool reports state, it does not make the determination. > [CmdletBinding()] param([switch]$SimReport, [string]$PolicyName, [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\IPMIG-Validation.ps1 -SimReport -PolicyName "Reclassify Sim - Confidential - Locations" powershell -ExecutionPolicy Bypass -File .\IPMIG-Validation.ps1 -SimReport # all reclassification policies Requires ExchangeOnlineManagement (Connect-IPPSSession). Role: Information Protection reader/admin. The detailed simulated match SET (per item) is shown in the Purview portal Auto-labeling > Simulation tab — this tool reports mode/status/config and points you there; it does not read item bodies. EXPECTED RESULTS Per policy: Mode (Test* = simulation PASS before enforcement), status, target label, rule SIT. If a policy shows Mode=Enable it is ENFORCING (reclassifying) — expected only after precision sign-off. .NOTES Purview — Sensitivity Label Migration. Read-only. Whether precision is acceptable and whether to enforce are data-owner/records decisions — this tool reports state, it does not make the determination. > [CmdletBinding()] param([switch]$SimReport, [string]$PolicyName, [string]$UserPrincipalName)